SYSTEM MAP · VERIFIED AGAINST THE RUNNING CODE

Book/Audio Suite

One local, zero-cost web app running two mirror-image pipelines: a PDF becomes a narrated audiobook, and a recording becomes a punctuated transcript. This map shows what a user does, what the code does underneath, and what still needs fixing.

9 python modules | **2** pipelines | **11** HTTP endpoints | **0** paid APIs | **0** tests

runs at **localhost:8000**

PART A What the user sees

The whole product is one page with two tabs. Everything below is reachable without a terminal — upload, wait, download.

TAB 1 Book → Audio

```
Open localhost:8000
│
├── [theme toggle] light / dark — remembered next visit, follows your OS by default
│
└── Tab: Book → Audio
    │
    ├── 1. Choose a PDF
    │      accepts text PDFs and scanned ones (scanned needs Tesseract — not installed yet)
    │
    ├── 2. Pick a voice
    │   ├── 47 English voices, each labelled with its personality
    │   │      e.g. Andrew "Warm, Confident"  ·  Neerja Expressive (Indian English)
    │   ├── ▶ Preview — hear one fixed line in that voice before committing
    │   └──   the preview honours the speed you picked, so it matches the real result
    │
    ├── 3. Set narration speed   Normal · −8% · −12% (default) · −18% · +8%
    │
    ├── 4. Choose the format
    │   ├── MP3 (default) — plays on any phone or speaker
    │   └── M4B — chapter marks + remembers your place, for audiobook apps
    │
    ├── 5. Title / Author (optional — falls back to the filename)
    │
    └── 6. Create audiobook ──► progress bar, live
        │
        ├── Extracting text  →  Detecting chapters  →  Synthesizing speech
        │                    →  Stitching audio     →  Writing text PDF*
        │
        └── Downloads
            ├── Full audiobook     one file, whole book
            ├── Chapters (zip)     one MP3 per detected chapter
            └── Text PDF           *only appears for scanned books
```

A short book takes seconds; a full-length one takes minutes, because each chunk of text is a separate rate-limited call to Microsoft's voice service.

TAB 2 Audio → Text

```
Tab: Audio → Text
│
├── 1. Choose an audio file
│       mp3 · wav · m4a · flac · ogg · opus · aac · mp4 · mov · webm
│
├── 2. Pick a model  (accuracy vs. speed)
│    ├── Tiny — fastest
│    ├── Base — balanced (default)
│    ├── Small — more accurate
│    └── Medium — slow, best
│
├── 3. Pick a language
│    ├── Auto-detect (default)
│    └── English · Hindi · Spanish · French · German · Italian · Portuguese · Japanese · Chinese
│
├── 4. Title (optional — names the downloaded files)
│
└── 5. Transcribe audio ──► progress bar
     │
     ├── Loading speech model  →  Transcribing audio  →  Writing transcript
     │       the bar sits at 25% through the whole transcription — see D-4
     │
     ├── Transcript appears inline, so you can read it without downloading
     │       silent audio says "No speech was detected" instead of failing
     │
     └── Downloads
         ├── .txt   plain transcript, paragraph breaks on real pauses
         ├── .md    same text + language, duration, model
         └── .json  every segment and word with start/end times
```

The first transcription downloads the speech model (~140 MB) and is slower; every one after that reuses it. Nothing is uploaded anywhere — transcription happens on this machine.

WHAT BREAKS, IN USER TERMS

BUG **A book can come out with dozens of one-page "chapters"**

Page numbers are mistaken for chapter headings, so the book gets sliced at almost every page. Affects the chapters zip and the M4B chapter marks.

BUG **Dashes are read as if the words were joined**

An em-dash in "precisely—having" becomes a hyphen, so the narrator says it as one compound word instead of pausing.

GAP **Refreshing the page loses your download links** — jobs live in memory only, so a restart erases them. There is also no history of past conversions.

> GAP **Scanned PDFs will fail today** — the OCR step needs Tesseract installed (`brew install tesseract`). The text-PDF output that depends on it has never run end to end.

PART B What the code does

One FastAPI process serves the page, owns a single job registry, and hands both pipelines to the same two-worker thread pool. The two features share every piece of machinery except the work itself.

```
app.py — FastAPI, one job model for both features
│
├── Static + metadata
│   ├── GET  /                      serves index.html
│   ├── GET  /voices                edge_tts.list_voices(), filtered to en-*, cached in memory
│   ├── GET  /sample?voice&rate     synthesizes one line, cached on disk by md5(voice|rate|text)
│   └── GET  /stt/status            probes faster-whisper + numpy + ffmpeg → {ready, detail}
│
├── Job creation  (identical shape, different work)
│   ├── POST /convert       .pdf   → Job(kind="tts") → _run_job      → pipeline.run_pipeline()
│   └── POST /transcribe    audio  → Job(kind="stt") → _run_stt_job  → stt.pipeline.transcribe_file()
│
├── Shared machinery
│   ├── _jobs: dict[str, Job]           in-memory registry   lost on restart (D-3)
│   ├── ThreadPoolExecutor(max_workers=2)
│   ├── jobs/<job_id>/                  upload + outputs on disk
│   └── Job(kind, status, stage, progress, error, …outputs)
│
├── Status
│   └── GET  /jobs/{id}             polled every 1.5s; response shape branches on kind
│
└── Downloads
    ├── GET /jobs/{id}/full                TTS  audiobook
    ├── GET /jobs/{id}/chapters.zip        TTS  zipped per-chapter mp3s (streamed, ZIP_STORED)
    ├── GET /jobs/{id}/textpdf             TTS  OCR text PDF
    └── GET /jobs/{id}/transcript.{fmt}    STT  txt | md | json
```

REQUEST LIFECYCLE — THE PART WORTH INTERNALIZING

```
browser                          FastAPI (async)              worker thread
   │                                  │                            │
   ├── POST /convert ─────────────────►│                            │
   │                                  ├── save upload to jobs/<id>/
   │                                  ├── _jobs[id] = Job(...)
   │                                  ├── run_in_executor ─────────►│ pipeline runs
   │◄──────────── {job_id} ────────────┤                            │ mutates job.stage
   │                                  │                            │ mutates job.progress
   ├── GET /jobs/{id} ─── every 1.5s ─►│ reads the same Job object  │
   │◄── {status, stage, progress} ────┤                            │
   │          …repeat…                │                            │ sets status="done"
   │                                  │                            │
   └── GET /jobs/{id}/full ──────────►│ FileResponse ──────────────┘
```

Why a thread pool and not async: run_pipeline() calls asyncio.run() internally.
That is illegal inside a running event loop, but perfectly safe on a pool worker,
which has no loop of its own. The same trick lets the blocking whisper call sit
next to the async web server without stalling it.

B.1 Book → Audio internals

Five stages behind one call. Each module is independently runnable with its own `__main__` smoke test.

```
pipeline.run_pipeline(pdf, out_dir, PipelineConfig)
│
├── [1] extractor.extract_pages()                          → Iterator[PageResult]
│       PyMuPDF text layer per page
│       └── page sparse AND has an image? → Tesseract OCR   (lazy import, optional)
│       └── one bad page yields PageSource.ERROR, never kills the run
│
├── [2] chapters.detect_chapters()                         → ChapterSpan[]
│   │   ├── doc.get_toc()  ← trusted first, exact (title, page)
│   │   └── heading heuristics  ← fallback  over-splits: see D-1
│   └── chapters.group_pages()                               → ChapterSegment[]
│
├── [2b] cleaner.clean_text()  per chapter, ordered pipeline
│       normalize_unicode → dehyphenate → strip_page_numbers
│       → strip_running_headers → reflow_paragraphs → collapse_whitespace
│       unicode map folds em/en-dash to "-"  → see D-2
│
├── [3] tts.chunk_text()                                   → TextChunk(index, text)
│   │     greedy sentence packing to ≤3000 chars; oversize sentence hard-split
│   └── tts.synthesize_book()                              → AudioChunk[]
│         asyncio + Semaphore(4) · per-chunk retry w/ backoff · 120s timeout
│         writes chunk_00000.mp3, chunk_00001.mp3, …
│         skip_existing=True ⇒ resume; failed chunk returns ok=False, never raises
│
├── [4] stitch.stitch()  system FFmpeg via subprocess, never pydub
│   ├── .mp3  → concat demuxer with -c copy      (lossless, no re-encode)
│   ├── .m4b  → concat + AAC re-encode + chapter marks from ffmetadata
│   ├── full/<title>.mp3        whole book
│   └── chapters/NN_Title.mp3   one file per chapter
│
└── [5] textpdf.write_text_pdf()   only when OCR actually ran
        PyMuPDF Story engine paginates the cleaned text
```

The one idea everything rests on: a chunk's position is frozen into its `index` before any network call, and that index becomes its filename. Async synthesis finishes chunks out of order; order is recovered by sorting filenames, never by completion time.

B.2 Audio → Text internals

A four-stage package that predates the web layer and stays independently runnable — `python -m stt.cli transcribe file.mp3` works with no server.

```
stt.pipeline.transcribe_file(audio, out_stem, transcriber=SHARED)
│
├── [1] ingest.load_audio()
│       shells out to FFmpeg → mono 16 kHz float32 numpy array
│       one decoder for every container, instead of many python codecs
│
├── [2] transcribe.Transcriber.transcribe()
│       faster-whisper (Whisper on CTranslate2), int8 on CPU
│       word_timestamps=True · vad_filter=True · beam_size=5
```

```
│       → TranscriptResult(segments[], language, duration, model)
│
├── [3] formatter.format_transcript()
│       new paragraph whenever the gap between segments ≥ 2.0s
│       tidies spacing and space-before-punctuation
│
└── [4] output.write_all()
        <stem>.txt · <stem>.md (+ language/duration/model header) · <stem>.json

stt/stream.py  live microphone, rolling partials   — built, NOT wired to the web app
```

HOW THE WEB LAYER WRAPS IT

```
_run_stt_job()  — the mirror of _run_job
│
├── lazy import stt.pipeline        ← stt imports numpy at module level, so a
│                                        top-level import would crash the whole app
│                                        (including TTS) when STT deps are absent
├── _get_transcriber(model)
│     └── one shared Transcriber, guarded by _stt_model_lock
│           weights load on first use, then stay resident for every later job
├── with _stt_run_lock:                ← 2 pool workers, 1 WhisperModel:
│       transcribe_file(...)             serialized, because the model is not
│                                        safe to drive concurrently
└── job.empty = not result.text.strip()  ← silence is a result, not an error
```

B.3 Load-bearing rules

Break one of these and something fails quietly rather than loudly. They are the reason the code looks the way it does.

RULE	WHERE	WHAT BREAKS IF IGNORED
Order comes from the chunk index, never completion order	`tts.py`, `stitch.py`	Chapters play in scrambled order
The whisper model is built once and reused	`app.py` `_get_transcriber`	~140 MB reload on every request
Transcription is serialized by a lock	`app.py` `_stt_run_lock`	Concurrent use of one WhisperModel; CPU thrash
`stt` is imported lazily	`app.py`	A missing STT dep takes down the TTS feature too
The filesystem is the checkpoint	`pipeline.py --resume`	A crashed 600-page book restarts from zero
Fail loud at boundaries, degrade gracefully mid-run	every stage	One bad page or chunk kills an hour of work
Stop before stitching if any chunk failed	`pipeline.py`	Audio silently missing from the middle of a book
FFmpeg streams; python never holds audio	`stitch.py`, `ingest.py`	Gigabytes of PCM in RAM on a long book

PART C What to change, by phase

Ordered by what hurts a real user first. Everything here is verified against the current code, not speculative.

PHASE 1 · CORRECTNESS — THE OUTPUT IS WRONG TODAY

D-1 **Chapter detection over-splits on page numbers**

`chapters.py:57` — the fallback patterns `^\s*\d{1,3}\s*$` and the lone-roman-numeral rule match bare page numbers, so a book with no embedded TOC gets one "chapter" per page, and titles absorb body text. **Fix:** require the heading to be a standalone short line with sparse surroundings, clean the captured title, and add a confidence gate that falls back to fixed-size "Parts" when detection looks unreliable.

D-2

Em/en-dashes are flattened into hyphens before narration

`cleaner.py:27` maps `—` and `–` to `-`, so a dash that signals a pause becomes a compound word. **Fix:** a `normalize_dashes()` step — em-dash to a comma pause, and a digit-aware rule so `1914–1918` reads "1914 to 1918" rather than becoming a pause.

D-8

The OCR path has never been executed

Tesseract is not installed, so scanned PDFs fail at extraction and `textpdf.py` — which only triggers when OCR ran — is unverified end to end. **Fix:** `brew install tesseract` plus `pytesseract` / `pillow`, then one real scanned book through the whole pipeline.

PHASE 2 · OPERABILITY — IT WORKS, BUT NOT FOR LONG

D-3

Jobs live only in memory

A restart erases every job, so finished work becomes unreachable even though the files still sit in `jobs/`. This bit during testing — a completed audiobook 404'd after a reload. **Fix:** write a `job.json` beside the outputs and rehydrate `_jobs` on startup.

D-5

`jobs/` grows without bound — already 664 MB across 15 jobs

Nothing ever deletes a finished job, and each one keeps the upload, every intermediate chunk, and every output. **Fix:** drop `_chunks/` after stitching (the config flag exists), plus a TTL sweep on startup.

D-4

Transcription progress is coarse

`transcribe_file()` exposes no progress hook, so the bar jumps 25% → 92% and sits still through the slowest part. **Fix:** iterate faster-whisper's segment generator inside `stt/transcribe.py` and report `segment.end / duration` — the TTS side already proves the pattern with its per-chunk `on_done` callback.

D-9

No tests, anywhere

Every stage is verified by hand today. The pure functions are the cheap win: the chunker's packing and hard-split, the cleaner's transforms, chapter grouping, and the formatter's paragraph rule are all deterministic and need no network.

PHASE 3 · FEATURES — THE SPECS ALREADY CALL FOR THESE

F-1 **Wire up live-microphone transcription**

`stt/stream.py` exists and the CLI can drive it; the web app cannot. Needs a WebSocket endpoint pushing rolling partials, and a record button in the STT tab. This is the last unbuilt piece of the STT spec.

F-2 **Subtitle export (.srt / .vtt)**

The timestamps are already written to the JSON — this is a formatter and a fourth download button, nothing more. Highest value per line of code on the list.

F-3 **A more expressive voice engine**

Confirmed by inspection: `edge_tts.Communicate` accepts only text, voice, rate, volume and pitch — no `express-as` style, so emotional range cannot be dialled in on the free endpoint. Voice choice is the only lever. Kokoro or Piper drop in behind the existing one-coroutine engine seam without touching the chunker or the concurrency harness.

PHASE 4 · PUBLIC DEPLOYMENT — NONE OF THIS EXISTS YET

P-1 **No auth, quotas, or upload limits.** Any caller can queue unbounded work and fill the disk. Fine on localhost, a blocker the moment it is exposed.

P-2 **A two-worker in-process pool is not a deployment.** Multi-user use needs a real queue and shared storage; the job model was written to be swapped without touching either pipeline.

P-3 **Edge-TTS at server scale is unproven.** It is a free undocumented endpoint; the concurrency cap that keeps one user safe may not survive many.

HOUSEKEEPING

D-7 **The venv's `pip` is broken**

Its shebang still points at the pre-move folder path, so `./.venv/bin/pip` fails. Use `./.venv/bin/python -m pip`, or rebuild the venv.